Week 1 Lab Report

TECHIN 510 · Spring 2026

Component A: Staff Interview

Interview Notes

Guest: Dorothy, Program Coordinator — Student Purchasing

Focus: Managing purchase requests for GIX programs.

Key Findings

- Volume: 20–150 purchase orders per month, sustained over 3 months
- Primary purchasing platform: Amazon; also uses Digikey and other vendors
- Current tool: Shared Excel spreadsheet on OneDrive, editable by all students
- Previously tried Google Forms — received too many incomplete or incorrect submissions
- Each order requires: Team #, CFO name, supplier, item name, quantity, price, purchase link, notes
- Approvals are batched once per week (every Monday at 1:00 PM)
- Every order requires instructor approval before Dorothy gives final admin approval
- Data must never be deleted — only archived for historical reference
- Needs CSV export functionality to continue using Excel for calculations and reporting
- Only the team CFO (not any student) may submit purchase requests
- Receipts and packing slips must be saved until verified in Workday (3 days to 2 weeks)
- Lost packages: cancel the original order and reorder
- Returns: go through Amazon's return process; refund credited back to team budget

Direct Quote

"I can't get away from Excel. I need a way to put everything into some database."

Surprises

- The approval process is two-step: instructor must approve first, then Dorothy (admin) approves
- Dorothy also manages back-order situations — she proactively contacts students when items are delayed and suggests switching to Amazon for faster delivery
- The system needs to handle multi-vendor tracking in a single unified view

Current Workaround — User Journey

Step	Action	Where Info Lives	What Could Go Wrong
1	Student fills in shared Excel on OneDrive	OneDrive Excel	Wrong team name, missing fields, accidental overwrites
2	Dorothy manually reviews each row	Excel spreadsheet	Rows missed, no notification when new entry added
3	Instructor approves or rejects (ad hoc)	Email / in-person	No tracking, easy to forget, no rejection reason recorded
4	Dorothy places order on Amazon/Digikey	Her browser history	Wrong item, budget not checked in real time
5	Item arrives; Dorothy confirms receipt	Memory / sticky notes	No formal confirmation record, easy to miss
6	Dorothy verifies in Workday system	Workday (separate)	Delay of 3 days–2 weeks; receipts can be lost

Problem Statement

"When Dorothy needs to manage and track purchase requests across multiple student teams and vendors, she currently manually edits shared Excel sheets on OneDrive and processes everything by hand, which causes incomplete submissions, no status visibility, budget overruns going undetected, and Dorothy spending excessive time chasing down information instead of processing orders."

Component B: Lab — AI Usage Log

Interaction 1 — Generate Initial App

Prompt

> *Build a purchase request tracking web app using Python 3.11+ and Streamlit. Use SQLite for persistent data storage. The app should have 4 pages: Submit Purchase Request (CFOs), Admin Dashboard (Dorothy), Received Confirmation, and Summary & Export.*

What AI Produced

Generated 13 files including app.py, 4 page files under pages/, a database module (prt/db.py), finance module, and UI helpers. Full working app with SQLite backend, color-coded status badges, and real-time budget checking.

First try: Yes — app ran successfully on first attempt with no errors.

Interaction 2 — Architecture Refactor (Three-Role System)

Prompt

> *Read all existing files. Merge all 4 separate page files into a single app.py using st.tabs(). Add a role selector in the sidebar with three options: Student (CFO), Instructor, Admin (Dorothy). Based on the selected role, show different tabs.*

What AI Produced

Restructured the entire app into one file with role-based navigation, separate render functions per role, and a sidebar role selector. Added session_state management so role persists across rerenders.

First try: No. The student page showed "No teams configured for this class" and blocked the form entirely. Fixed by removing the outdated team validation check since Team Number was changed to a free text input.

Interaction 3 — Login System + UW Branding

Prompt

> *Add UW banner (purple #4b2e83, gold border), login/register page with Sign In and Create Account tabs, three roles, password hashing with hashlib sha256, store user in session_state, auto-set role after login, pre-populate admin@uw.edu account.*

What AI Produced

Full authentication system with a users table in SQLite, login wall before any content is shown, role-based automatic page routing, Sign Out button in sidebar, and a pre-populated admin@uw.edu account.

First try: No. The UW banner CSS caused a rainbow gradient artifact at the top of the page and the banner itself disappeared after CSS conflicts with Streamlit's internal styles. Fixed by

replacing the CSS-based approach with an inline HTML div that renders reliably across all page states.

Interaction 4 — Weekly Submission Window System

Prompt

> *Change the deadline system completely. Dorothy sets weekly deadlines (every Monday at 1:00 PM) in the Admin Dashboard. Remove the deadline date picker from the student submit page — students cannot choose their own deadline. Show a countdown to the current window on the student page. Auto-assign orders to the current active window. If the window is closed, assign to the next Monday 1PM window and notify the student. Auto-generate 8 weekly windows when a new class is created.*

What AI produced: Added a submission_windows table to the SQLite database. Built a weekly window management UI in the Admin Dashboard where Dorothy can activate or deactivate each window. Replaced the student deadline picker with a real-time countdown showing days and hours remaining. Orders are automatically assigned to the correct window on submission. If the window has already passed, the system assigns the order to the next Monday and shows the student a message explaining this.

First try: No. The countdown displayed the wrong time because the system was comparing naive datetime objects with timezone-aware ones. Fixed by ensuring all datetime comparisons use timezone-aware objects with .astimezone(). Also, the auto-generated windows were initially using UTC instead of local time, causing the 1PM deadline to appear at the wrong hour for users in the Pacific timezone.

Accessibility Baseline Results

Color Contrast

- Result: PASS
- UW purple (#4b2e83) on white background: contrast ratio ~9.7:1, well above the 4.5:1 WCAG requirement
- Status badges: green (#15803d) on light green (#dcfce7), amber (#d97706) on light yellow (#fef3c7), red (#dc2626) on light red (#fee2e2) — all pass

Semantic Headings

- Result: PASS
- st.title() used once per page for the main app title "Purchase Request Tracker"
- section_header() renders <h2> for all page titles (Submit Purchase Request, Admin Dashboard, Instructor)
- ##### markdown renders <h5> for subsections (Budget check, Line items, Submission window)

- No heading levels are skipped; structure follows a logical hierarchy

Reflection

What surprised you about AI-assisted coding?

The AI could read the entire codebase context and make coordinated changes across multiple functions simultaneously without losing track of the overall architecture. When I asked it to refactor the role system, it correctly updated the database schema, the UI rendering functions, the sidebar logic, and the session state management all in one pass — far more capable than I expected from a code generator.

What did the AI get wrong?

Several things required multiple rounds of fixing. CSS selectors were frequently too broad — a border-left accent meant for the budget card spread across the entire main content column, creating a thick purple line running down the whole page. Streamlit's st.expander() cannot be nested inside another st.expander(), which the AI attempted when building the admin grouped order view, causing the app to crash on load. Datetime comparison bugs caused the weekly submission window countdown to display incorrect times because the AI mixed naive and timezone-aware datetime objects. The AI also initially placed the "Add Another Item" button above the item list instead of below it, which is counterintuitive — users expect to see the existing items first, then the option to add more. Each of these required targeted follow-up prompts to fix rather than regenerating the entire file.

Could you explain your code?

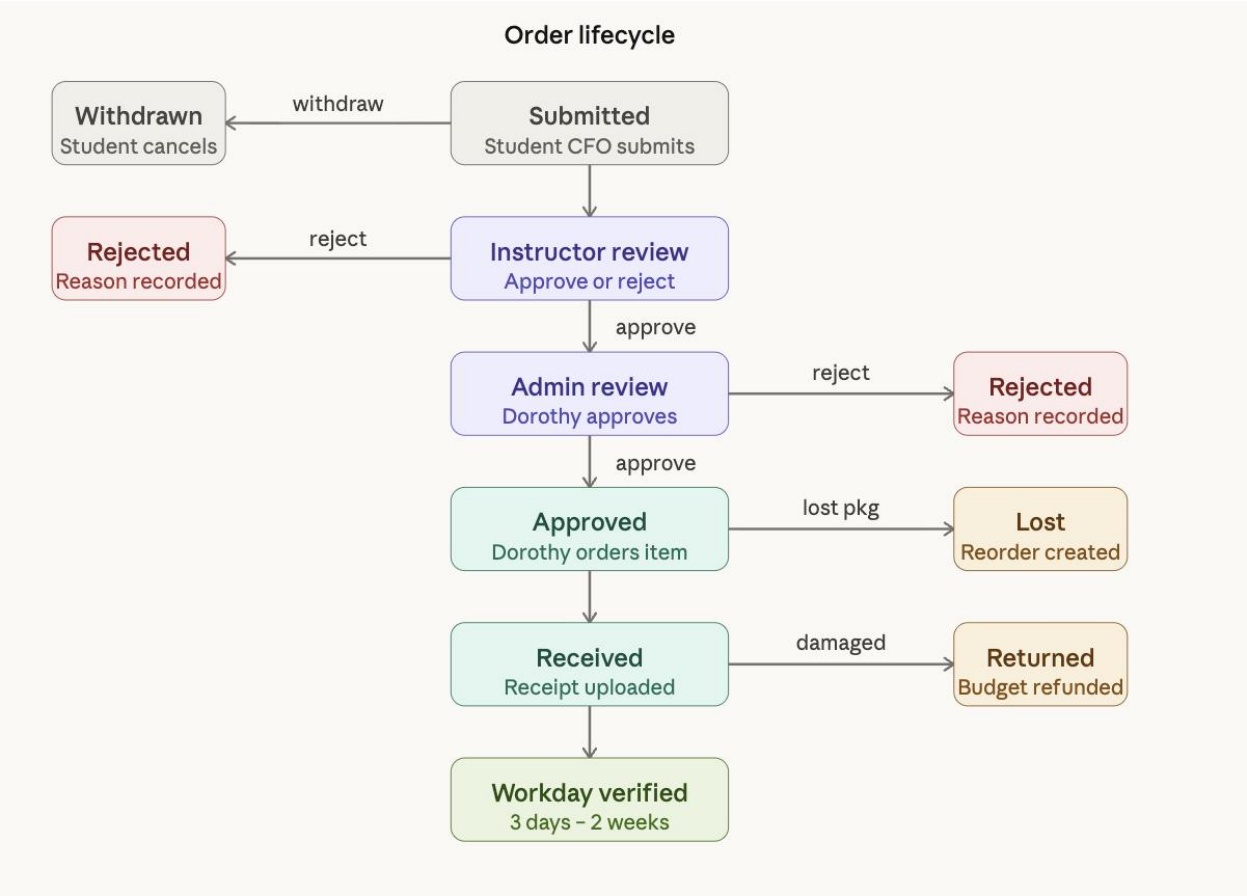
Yes for the high-level structure. Each render_*() function handles one role's UI independently. Database calls are abstracted into prt/db.py so the UI layer never touches SQL directly. Streamlit session_state carries user identity and active class selection across rerenders. The budget enforcement checks remaining = total_budget - approved_orders before allowing submission, and blocks the Submit button if the total would exceed the remaining balance.

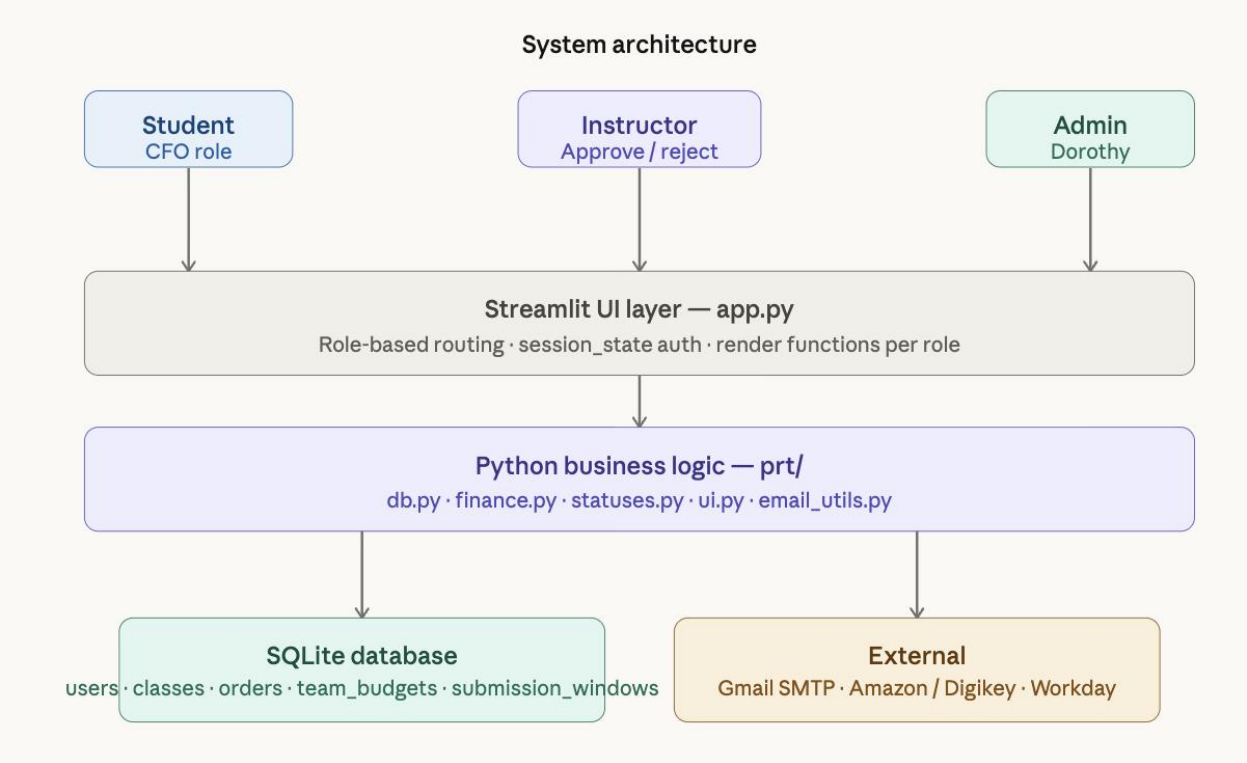
What did you learn from the interview?

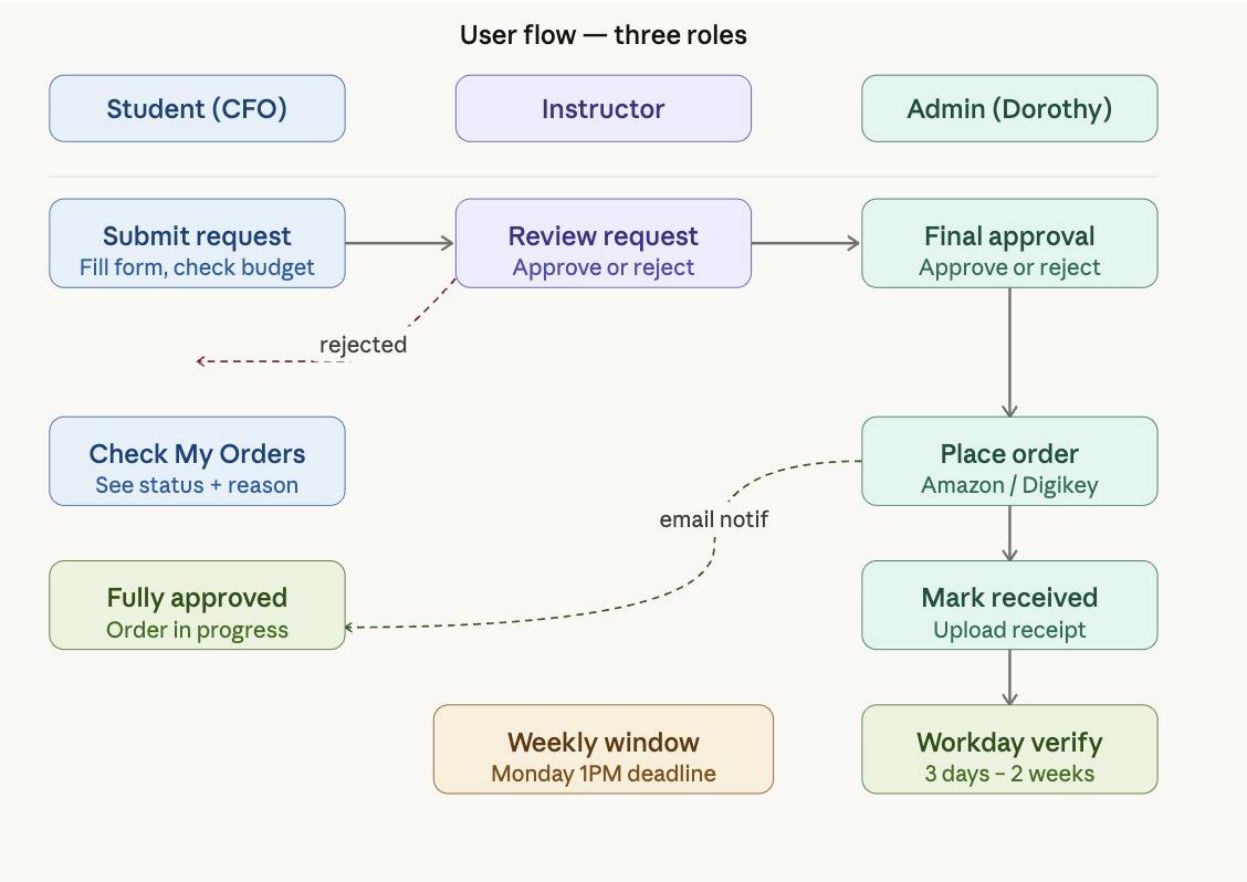
Without talking to Dorothy, I would have built a simple form with a submit button. The interview revealed the real operational complexity: weekly approval cycles locked to Monday 1PM, a two-step approval chain (instructor then admin), multi-vendor tracking, per-team budget enforcement, receipt archiving tied to Workday verification, and the need for CSV export so Dorothy can continue her existing Excel workflows. None of this would have been obvious without listening to her describe her actual workday.

Component C: System Architecture & Design

Input / Process / Output Diagram




System architecture
Student
CFO role
Instructor
Approve / reject
Admin
Dorothy
Streamlit UI layer — app.py
Role-based routing · session_state auth · render functions per role
Python business logic — prt/
db.py · finance.py · statuses.py · ui.py · email_utils.py
SQLite database
users · classes · orders · team_budgets · submission_windows
External
Gmail SMTP · Amazon / Digikey · Workday


User flow — three roles
Student (CFO)
Instructor
Admin (Dorothy)
Submit request
Fill form, check budget
Review request
Approve or reject
Final approval
Approve or reject
rejected
Check My Orders
See status + reason
Place order
Amazon / Digikey
email notif
Fully approved
Order in progress
Mark received
Upload receipt
Weekly window
Monday 1PM deadline
Workday verify
3 days – 2 weeks

↺ Feedback Loop:

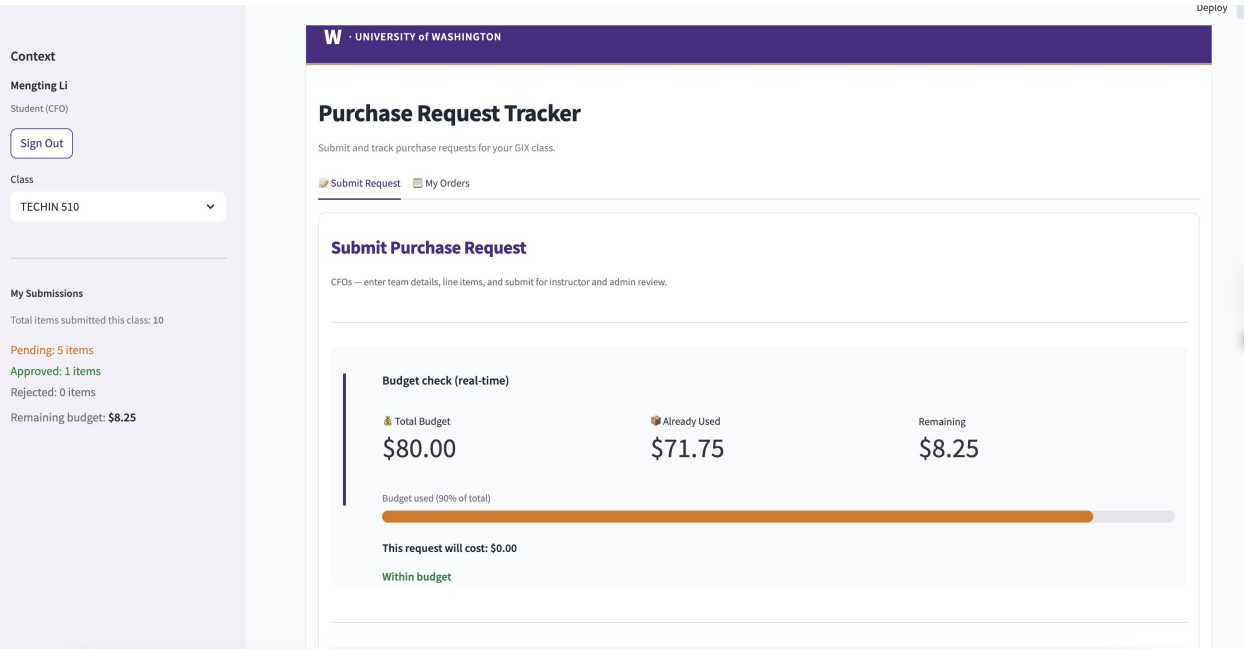
Student sees rejection reason → revises item and resubmits, or withdraws the request entirely. Dorothy approves → auto-email notification sent to student. Dorothy marks received and uploads receipt → order moves to Workday verification pending. If package is lost, Dorothy triggers Cancel & Reorder which creates a replacement order starting the approval flow from the beginning. Summary report updates spend totals and flags any teams close to or over budget.

Design Decision Log

Field	Entry
Decision	Put all page logic in one file (app.py) instead of separate page files under pages/
Alternatives considered	1) Keep 4 separate files under pages/ (original Streamlit multipage structure) 2) Split into app.py + one module per role (e.g., student_view.py, admin_view.py)
Why you chose this	Streamlit's multipage system (pages/ folder) does not support role-based access control — any user can navigate directly to any page URL regardless of their login role. A single file with conditional rendering based on session_state.user_role enforces access cleanly and ensures students can never reach the Admin Dashboard by typing a URL.
Trade-off	app.py became very long (~1,500+ lines), harder to navigate and search. A monolithic file is harder for multiple developers to work on simultaneously without merge conflicts.
When would you choose differently?	If the team grows beyond 2 developers, or if the app scales beyond 5 distinct roles. In that case, splitting into role-specific modules with a shared authentication middleware (checking session_state at the module level) would be worth the added complexity.

Component D: Testing & Validation

App Screenshot



Smoke Test Results

#	Feature Tested	Action Taken	Expected Result	Actual Result	Pass/Fail
1	Login authentication	Entered admin@uw.edu with correct password	Redirect to Admin Dashboard; sidebar shows Dorothy's name and role	Redirected correctly; sidebar showed "Dorothy / Admin" with Sign Out button	Pass
2	Budget enforcement	As student, entered unit price $100 with only $36.10 remaining budget	Submit button disabled; red over-budget warning shown	Submit button greyed out; warning: "Over budget by $63.90 — please reduce your order"	Pass
3	Empty field validation	Left Item Name blank, filled all other fields, clicked Submit	Error: "Item 1 — Item Name is required"	Red error displayed; form did not submit and data was not written to database	Pass
4	Submission window countdown	Opened student submit page on April 5, 2026	Current window displayed with accurate time remaining	Showed "Current window: Monday April 06, 1:00 PM" with "1 day 11 hours remaining" in green	Pass

#	Feature Tested	Action Taken	Expected Result	Actual Result	Pass/Fail
5	Two-step approval flow	As Instructor, approved a pending student order	Order status changes; Admin Dashboard shows instructor approval badge	Status updated to PROCESSING instantly; Admin table showed √ Instructor badge in green	Pass

Quality Gate Checklist

- Smoke test table completed with 5 tested features
- No failed tests — all 5 features passed on first test run
- Screenshot of running app captured (submit separately with repo)
- Accessibility baseline results recorded: color contrast PASS, semantic headings PASS

Edge Cases Identified

Edge Case 1: Over-budget submission attempt

Why it matters: Without budget enforcement, a student could accidentally submit an order that exceeds their team's allocated budget. This would require Dorothy to manually calculate whether the total spend is within limits — exactly the kind of manual work the system is designed to eliminate. The system enforces this by calculating remaining = total_budget − approved_orders and comparing against the requested total before allowing submission. The Submit button is disabled client-side and the check is repeated server-side on submit to prevent bypass.

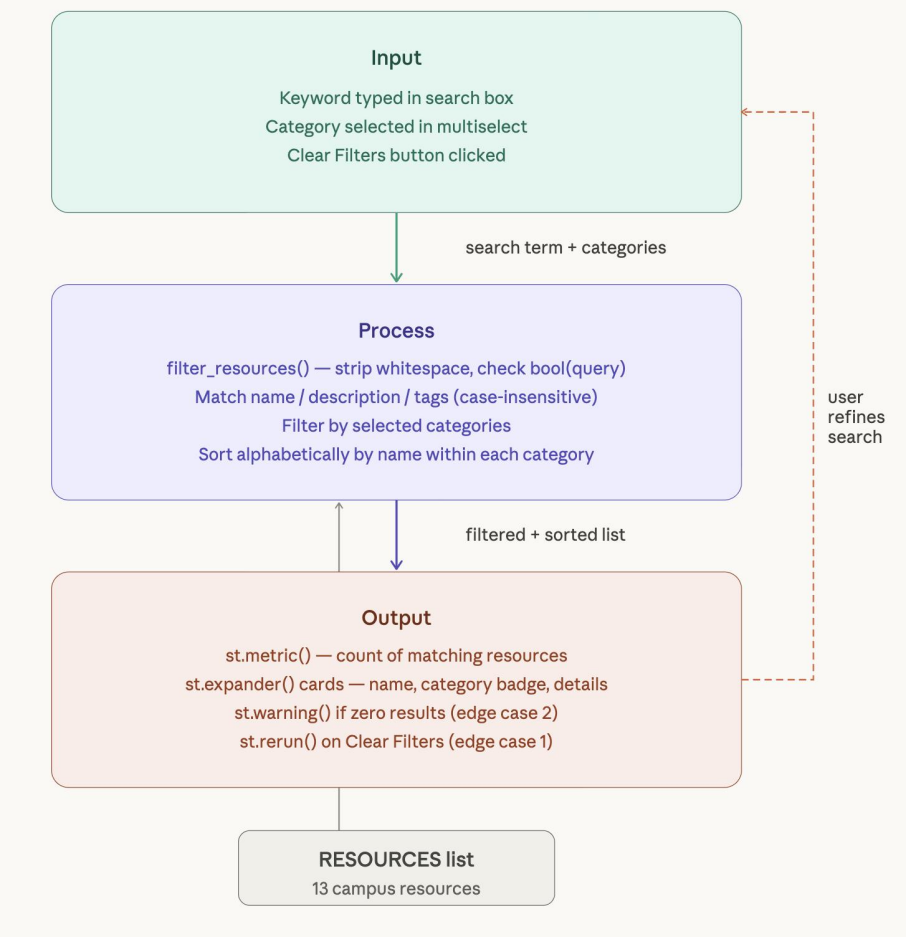
Edge Case 2: Submission after window deadline

Why it matters: If a student submits after Monday 1PM, their order should not be silently lost or added to the closed week. The system detects whether the current time is past the active window deadline and automatically assigns the order to the next Monday 1PM window, showing the student a clear message: "The current window is closed. Your request has been added to next week's purchasing list." This prevents confusion and ensures Dorothy always sees orders grouped correctly by their intended purchase week.

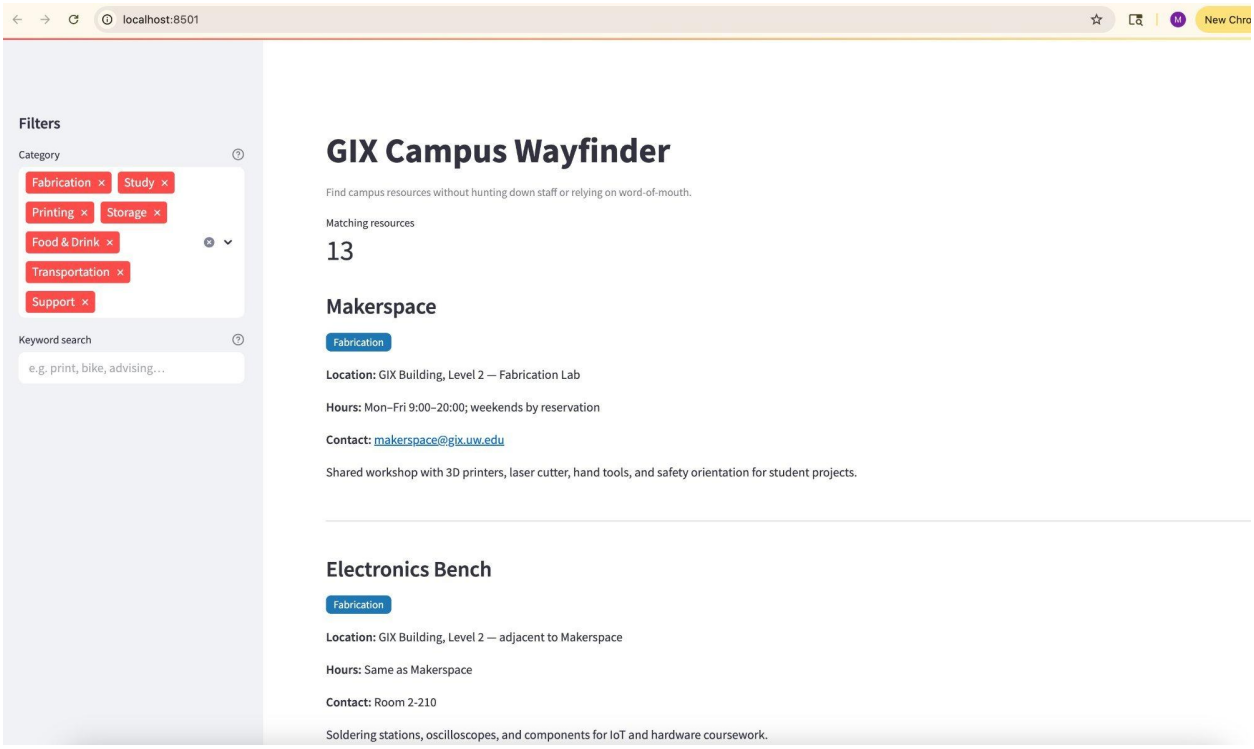
Component E: GIX Campus Wayfinder

I/P/O Architecture Diagram

The GIX Campus Wayfinder follows a three-stage Input → Process → Output architecture with a feedback loop from output back to input.



App Screenshot



Edge Case Validation

Edge Case 1: Empty or whitespace-only search input

Why it matters: This is the most common user action after opening the app. When a user clears the search bar or leaves it blank, a naive implementation might match the empty string against all resources and show everything — which happens to be correct by accident — but whitespace-only input (e.g. the user pressed spacebar) would silently show zero results and leave the user thinking the app is broken. The filter_resources() function explicitly strips whitespace with .strip() and checks bool(query) before applying any text filter. This guarantees that both empty and whitespace-only inputs are treated as “no filter,” showing all resources in the selected categories.

Test performed: Typed “ ” (three spaces) into the search box. Expected result: all 13 resources shown. Actual result: all 13 resources shown. Pass.

Edge Case 2: Search term with no matching resources

Why it matters: When no resources match the user’s search, the filtered list is empty. Without explicit handling, Streamlit renders a blank white area below the metric card — no error, no message, just silence. A user seeing a blank page has no idea whether the app crashed, the data is gone, or their search simply returned nothing. The code checks if len(sorted_matches) == 0 and calls st.warning(“No resources found. Try a different search term or category.”) This gives users a clear signal and a path forward without crashing the app.

Test performed: Typed “xyzabc” into the search box. Expected result: warning message shown, no crash. Actual result: yellow warning banner “No resources found. Try a different search term or category.” displayed. Pass.

Assert Statement for Data Integrity

Location in code: app.py, lines immediately after the RESOURCES list definition.

```
required_fields = [“name”, “category”, “location”, “hours”, “description”, “tags”, “contact”]
for resource in RESOURCES:
    for field in required_fields:
        assert field in resource and resource[field], (
            f“Resource missing or empty field: ‘{field}’ in {resource.get(‘name’, ‘unknown’)}”
        )
assert len(RESOURCES) >= 12, “Must have at least 12 resources”
```

This assert block runs every time the app starts. It verifies that every resource in the RESOURCES list contains all 7 required fields and that none are empty. It also confirms a minimum dataset size of 12 resources. If any resource is malformed, the app fails immediately with a descriptive error identifying exactly which resource and which field caused the problem — rather than silently displaying incomplete data to users.

AI Prompt Log

Initial Prompt

> *“Build a Streamlit app called GIX Campus Wayfinder that helps new GIX students find campus resources without needing to ask staff or relying on word-of-mouth. Create a RESOURCES list of dictionaries with fields: name, category, location, hours, description, tags, contact. Populate with at least 12 resources across 7 categories. Add sidebar filters for category and keyword search. Display each resource as a card. Handle edge cases for empty search and no matches. Include an assert block verifying all required fields are present and non-empty.”*

Result: AI generated a complete working app with all 13 resources, filter logic, and the assert block in one pass. App ran successfully on first attempt.

Refinement Prompt

> *“The app works, but I want to improve usability. Please make these changes: (1) Add a Clear Filters button in the sidebar that resets both the search input and category multiselect. (2) Show the category as a colored badge next to each resource name using a different color per category. (3) Sort results alphabetically by name within each category group. (4) Add an st.expander() around each resource card so users can expand/collapse details. Do not change the data structure or the assert statement.”*

What changed and why: The initial version displayed all 13 resources fully expanded at all times, requiring excessive scrolling. There was also no way to reset filters without manually deselecting each category. The refinement added st.expander() cards (first 3 open, rest collapsed), a Clear Filters button using st.session_state and st.rerun(), color-coded badges for

visual category scanning, and alphabetical sorting within groups. These changes significantly reduced cognitive load and improved navigation speed.